PRODUCT OVERVIEW · FOR DEVOPS AND PLATFORM TEAMS

GPU Cost Optimizer

Find the GPUs you pay for but don't use — priced in ₹ at your own rates, from inside your own Kubernetes cluster.

What it does

GPU Cost Optimizer runs inside your Kubernetes cluster and watches how every GPU is actually used, hour by hour, and which workload is using it. It tells you which GPUs are wasting money, how much in rupees, and what to do about each one. **It observes and recommends; it never changes anything in your cluster.**

What it finds

Finding	What it means	What you see
Idle GPUs	A GPU allocated to a workload but essentially unused for hours.	₹ already spent on the idle time
Oversized GPUs	A workload whose load never needs the GPU it has — including LLM inference servers that look busy at peak but are quiet on average.	₹ per month saved by a smaller GPU
Recurring quiet hours	The same hours of the day are quiet on most days.	₹ per month saved by scaling down in those hours
Review candidates	Underused GPUs on one node that might share hardware; multi-GPU jobs whose GPUs are mostly idle.	Upper bound, for your team to review
Unexplained under-use	Below-target GPU time nothing above explains, including GPUs with no pod at all.	₹ already spent, no recommendation yet

Recommendations your team can judge

- Every “move to a smaller GPU” recommendation is checked against the three things a workload can run out of: **compute, memory bandwidth and memory capacity**.
- Each one states its cost, e.g. “each request may take up to ~1.6x longer — fine for batch work, check latency targets before moving an interactive service.”
- Where memory fit can't be determined (servers such as vLLM pre-allocate GPU memory), the recommendation says so instead of guessing.
- The headline ₹ figure counts each GPU once.

How it runs in your cluster

- **Install:** one Helm chart into one namespace, about 15 minutes. One small pod (about 0.2 CPU and 0.6 GiB memory requested) plus standard exporters.
- **Read-only:** Kubernetes access is limited to get, list and watch on pods, nodes and namespaces. Nothing can create, change or delete.
- **Your data stays with you:** metrics and findings are stored on a volume in the product's namespace. Nothing is sent outside your cluster, except an optional alert webhook you configure yourself.
- **Standard GPU metrics:** uses NVIDIA's DCGM exporter. Already running it (e.g. through the GPU Operator)? The product reuses yours and deploys no privileged pod.
- **Hardened:** containers run as non-root with read-only file systems; the report is reached through kubectl port-forward with a bearer token.
- **Uninstall:** two commands, nothing left behind.

The first cluster test (3–4 weeks)

Day 0	Install (about 15 minutes) and confirm every GPU is being read.
Days 1–7	The product collects data. Nothing for your team to do.
Week 2	Review call: every finding marked useful, wrong or already known.
Weeks 2–3	Your team acts on one recommendation of its choice.
Week 4	Measured saving and latency compared with what was predicted.

WHAT COMES NEXT

Automated recommendation actions

Once a client cluster test confirms that the recommendations are accurate on a real cluster, we will add the option for the product to carry out recommendations for you, instead of your team applying each one by hand:

- Reclaim idle GPU allocations.
- Scale workloads down during their recurring quiet hours, and back up after.
- Move a workload to the recommended smaller GPU.

Automation will be opt-in for each type of action, show exactly what will change before it runs, and wait for your approval. It needs write access, so it will be a separate component — the read-only product stays as it is.